# سند شفافیت استفاده از هوش مصنوعی

این سند بر اساس بند ۷-۳ صورت پروژه تدوین شده است. هدف آن، تفکیک صریح آنچه با کمک ابزار مولد تولید شده از آنچه دانشجو خودش نوشته، بازبینی یا اصلاح کرده است.

> **بخش‌های علامت‌خورده با `<<< ... >>>` را خود دانشجو باید پر کند.** این بخش‌ها درباره رفتار شخص دانشجو هستند و ابزار مولد نمی‌تواند آن‌ها را به جای او بنویسد.

---

## ۱. ابزارها و مدل‌های مورد استفاده

| ابزار | مدل | نقش در پروژه |
|---|---|---|
| Claude Code (رابط خط فرمان) | Claude Opus 5 | تولید ساختار پروژه، لایه دامنه، رابط کاربری، محتوای پرونده تحلیلی، آزمون‌ها و مستندات |
| `<<< اگر ابزار دیگری استفاده کرده‌اید اینجا اضافه کنید >>>` | — | — |

کد نوشته‌شده در همین مخزن هستند. قالب آماده یا فروشگاه‌ساز استفاده نشده است؛ تمام فایل‌های `src/`، CMS هیچ.

## ۲. نوع دسترسی و حساب کاربری

Claude Code در رابط خط فرمان Claude **اشتراک پولی** — دسترسی از طریق حساب اشتراکی.

## ۳. استراتژی هدایت و مهندسی پرامپت

روش کار در این پروژه «پرامپت تک‌مرحله‌ای» نبود. ترتیب هدایت به این شکل بود:

1. **تثبیت منبع الزامات:** فایل صورت پروژه درس به عنوان مرجع اصلی به ابزار داده شد و خواسته شد الزامات **فقط فاز ۱** از آن استخراج شود (بخش الف، بخش ب، و موارد تکمیلی و مزیت‌بخش).
2. **تثبیت هویت و دامنه کسب‌وکار:** سند معرفی تارنو (نام، شعار، گروه‌های هدف، سه تمایز اصلی) به عنوان ورودی دوم داده شد.
3. **تعیین مرز فاز:** صریحاً تأکید شد که بوم مدل کسب‌وکار، قیف فروش، پرسونا، ۱۰ سناریو و درگاه پرداخت واقعی **نباید** ساخته شوند، چون به فازهای ۲ تا ۴ تعلق دارند.
4. **قاعده صداقت در تحلیل:** خواسته شد هیچ آمار بازاری بدون منبع ذکر نشود، هیچ قابلیتی برای رقبا ساخته یا نفی نشود، و نتیجه مصاحبه‌ای که انجام نشده به عنوان یافته ارائه نگردد.
5. **ترتیب ساخت:** ابتدا لایه دامنه (تایپ‌ها، اعتبارسنجی، موتور موجودی، سرویس‌ها)، سپس آزمون‌های آن، و تنها پس از سبز شدن آزمون‌ها، لایه رابط کاربری.
6. **بازبینی تکرارشونده:** پس از هر مرحله، بررسی تایپ (`tsc --noEmit`)، ساخت (`vite build`) و اجرای آزمون‌ها انجام و خطاها اصلاح شد.

## ۴. تصمیمات معماری متأثر از مدل

تصمیم‌های زیر با همفکری ابزار گرفته و در `docs/ADR.md` مستند شده‌اند:

| انتخاب و دلیل | گزینه‌های بررسی‌شده | تصمیم |
|---|---|---|
| **لایه دامنه مستقل** — تا منطق کسب‌وکار بدون مرورگر قابل آزمون باشد | منطق درون کامپوننت‌ها / لایه دامنه مستقل | جداسازی منطق از رابط کاربری |
| **تأیید خریدار** — در غیر این صورت چند درخواست هم‌زمان می‌توانستند موجودی را بی‌دلیل قفل کنند | هنگام پذیرش فروشنده / هنگام تأیید خریدار | نقطه رزرو موجودی |
| **دفتر سه‌بخشی** — تنها راه پشتیبانی درست از فروش جزئی و جلوگیری از بیش‌فروشی | یک عدد «موجودی» / دفتر سه‌بخشی با ناوردا | نگهداری موجودی |
| اجرا روی هر میزبان ایستا بدون تنظیم `HashRouter` — rewrite | `BrowserRouter` / `HashRouter` | نوع مسیریابی |
| **وابسته به دسته** — چون الزام فنی پارچه مبلمان با پوشاک یکسان نیست | فرم ثابت / فیلدهای وابسته به دسته | فیلدهای فنی آگهی |
| **با درایور جداشده** — استقرار بدون **LocalStorage** سرور، با آمادگی مهاجرت | LocalStorage / پایگاه داده واقعی ماژولار | ذخیره‌سازی |
| **داده ساختاریافته** — تا همان محتوا به کامپوننت‌های تعاملی و نمودار تغذیه شود | داده ساختاریافته / JSX متن ایستا در `lib/content` | قالب محتوای پرونده |

## ۵. اصلاح، بازنویسی و دیباگ — خطاهایی که واقعاً رخ داد

این بخش گزارش دقیق خطاهایی است که در همین نشست تولید رخ داد و برطرف شد. ثبت آن‌ها برای شفافیت علمی است، نه برای اثبات بی‌خطا بودن ابزار:

| # | خطا | علت | اصلاح انجام‌شده |
|---|---|---|---|
| ۱ | شکست `T-15` و `T-11` دو آزمون خوردند | داده پایه از قبل یک درخواست باز برای همان خریدار روی همان آگهی داشت و قاعده «ضد درخواست تکراری» درست عمل می‌کرد؛ **فرض آزمون غلط بود، نه کد** | آزمون‌ها روی آگهی بدون درخواست باز منتقل شدند و یک آزمون تازه (`T-11ب`) برای سنجش همان قاعده اضافه شد |
| ۲ | کلاس‌های شفافیت رنگ (مثل `border-danger/40`) در خروجی CSS تولید نمی‌شدند | رنگ‌های Tailwind در `tailwind.config.js` به صورت `var(--x)` تعریف شده بودند و Tailwind نمی‌تواند روی مقدار خام متغیر، شفافیت اعمال کند | رنگ‌ها به تابع `color-mix` تبدیل شدند تا در صورت وجود شفافیت از آن استفاده کنند |
| ۳ | اجراکننده آزمون با `node --experimental-strip-types` کار نکرد | نسخه Node نصب‌شده بدون پشتیبانی TypeScript ساخته شده بود | تغییر کرد: `esbuild` اجراکننده به Node ابتدا بسته‌بندی، سپس اجرا با |
| ۴ | چند خطای بررسی تایپ | ایمپورت بی‌استفاده، پارامتر بی‌استفاده و ارجاع به فایل ساخته‌نشده | `tsc --noEmit` رفع و اجرای مجدد تا رسیدن به خروجی بی‌خطا |
| ۵ | نصب وابستگی‌ها چند بار با خطای شبکه | سرعت پایین اتصال | نصب با تکیه بر کش محلی و npm دریافت تک‌به‌تک بسته‌های کم‌شده تکمیل شد |

| # | خطا | علت | اصلاح انجام‌شده |
|---|---|---|---|
| | (ERR_SOCKET_TIMEOUT) قطع شد | | |
| ۶ | حذف هم‌زمان چند رکورد «نمونه قالب» تنها یک رکورد را پاک می‌کرد | لایه مخزن وضعیت، هر فراخوانی همان رندر اجرا db سرویس را روی می‌کرد؛ چند فراخوانی پشت‌سرهم در یک رویداد، همه روی وضعیت کهنه عمل می‌کردند | یک ارجاع به StoreProvider در آخرین وضعیت اضافه شد تا هر فراخوانی روی نتیجه فراخوانی قبلی اعمال شود. **این نقص را آزمون UI-13 پیدا کرد.** |
| ۷ | مقدار فیلدهای فرم مصاحبه در آزمون ثبت نمی‌شد | فرم از به‌روزرسانی غیرتابعی وضعیت استفاده می‌کرد (setForm({...form, x})) و تغییرهای پشت‌سرهم یکدیگر را بازنویسی می‌کردند | همه به‌روزرسانی‌ها به شکل تابعی (setForm((f) => …)) بازنویسی شدند |
| ۸ | کادرهای فیلتر عددی بازارگاه با «حذف همه فیلترها» خالی نمی‌شدند | ورودی‌ها کنترل‌نشده بودند و تنها مقدار پارس‌شده در وضعیت فیلتر نگه داشته می‌شد | متن خام هر ورودی هم در وضعیت نگه داشته شد تا کادرها واقعاً بازنشانی شوند |
| ۹ | دو آزمون یکپارچه سراغ عنصر نادرست می‌رفتند | متن دکمه ارسال زیررشته متن پیوند ناوبری بود (مثلاً «ثبت آگهی» در برابر «ثبت آگهی جدید») | برای تطبیق exactButton کمک‌کار دقیق متن اضافه شد |
| ۱۰ | رویداد input در jsdom به سامانه رویداد React نمی‌رسید | محدودیت محیط آزمون، نه نقص محصول (کلیک به درستی می‌رسید) | نمونه‌ساز نوشته و setter مقدار با ثبت‌شده روی گره onChange سپس فراخوانی می‌شود |

## ۶. آزمون و راستی‌آزمایی انجام‌شده

| مورد | وضعیت واقعی |
|---|---|
| npm run test:domain | اجرا شد — **۳۷ آزمون، ۳۷ موفق** |
| npm run test:render | اجرا شد — **۲۴ رندر مسیر، همه سالم** |
| npm run test:ui | اجرا شد — **۱۸ آزمون یکپارچه روی DOM واقعی (jsdom)، ۱۸ موفق** |
| npx tsc --noEmit | اجرا شد — بدون خطا |
| npm run build | اجرا شد — ساخت موفق (dist/) |
| آزمون دستی مرورگر | **مورد اجرا نشد**: چیدمان در عرض موبایل و ترتیب فوکوس صفحه‌کلید. این دو به موتور چیدمان و ۲ قابل سنجش نیستند؛ در jsdom مدیریت فوکوس مرورگر واقعی نیاز دارند و با با برچسب «اجرا نشده» ثبت شده‌اند docs/TEST_RESULTS.md |
| مصاحبه میدانی | **انجام نشد**؛ در پرونده هم صریحاً اعلام شده است |

هیچ آزمونی که اجرا نشده باشد، «موفق» علامت نخورده است.

## ۷. آنچه دانشجو باید خودش تکمیل کند

1. `<<< بخش‌هایی از کد که خودتان بازنویسی یا تغییر دادید را اینجا فهرست کنید >>>`
2. `<<< دو آزمون دستی باقی‌مانده (فوکوس صفحه‌کلید T-30 چیدمان موبایل و T-28) را در docs/TEST_RESULTS.md ثبت کنید >>>`
3. `<<< مصاحبه‌های میدانی را انجام دهید و پاسخ‌ها را در ابزار اعتبارسنجی سامانه وارد کنید >>>`
4. `<<< کار راستی‌آزمایی رقبا را انجام دهید و خانه‌های «نامعلوم» ماتریس مقایسه را به‌روز کنید >>>`

## ۸. محدودیت‌ها و مسائل باقی‌مانده

- **تسلط فردی:** طبق صورت پروژه، مسئولیت تک‌تک خطوط کد بر عهده دانشجو است. پیش از جلسه تحویل، خواندن و درک کامل این چهار فایل ضروری است، چون قلب منطق سامانه‌اند: `src/lib/domain/inventory.ts`، `services.ts`، `validation.ts` و `transitions.ts`.
- **نقش آزمون در این پروژه:** سه نقص واقعی محصول (ردیف‌های ۶ تا ۸ جدول بالا) را آزمون‌ها پیدا کردند، نه بازبینی چشمی. این نکته برای پاسخ به پرسش ارزیاب درباره «چرا آزمون نوشتید؟» قابل استناد است.
- **بسته جاوااسکریپت حدود ۵۸۵ کیلوبایت** است (حدود ۱۸۷ کیلوبایت فشرده). برای یک پروژه درسی قابل قبول است؛ بهینه‌سازی با تکه‌تکه کردن مسیرها در فازهای بعد.
- **تصاویر پارچه شبیه‌سازی‌شده‌اند** (تولیدشده SVG) و در رابط کاربری با برچسب «نمونه بافت شبیه‌سازی‌شده» مشخص شده‌اند؛ تصویر واقعی پارچه در اختیار نبود.
- **داده رقبا کامل نیست:** بخش بزرگی از ماتریس مقایسه عامدانه «نامعلوم» است، چون راستی‌آزمایی میدانی انجام نشده.
- **پرداخت کاملاً ساختگی است** و هیچ اتصال بانکی وجود ندارد.